Reviving, reproducing, and revisiting Axelrod's second tournament

Vincent Knight[1, *], Owen Campbell[2], Marc Harper[3], T. J. Gaffney[4], and Nikoleta E. Glynatsi[5, 6]

[1]School of Mathematics, Cardiff University, Cardiff, United Kingdom

[2]Independent Researcher, Cardiff, United Kingdom

[3]Google Inc., Mountain View, California, USA

[4]Independent Researcher, Nevada, USA

[5]Discrete Event Simulations Teams, RIKEN Center for Computational Science, Kobe, Japan

[6]Mathematical Social Science Team, RIKEN Center for Interdisciplinary Theoretical and Mathematical Sciences, Wako, Japan

[*]Corresponding author: KnightVA@cardiff.ac.uk.

Abstract

Direct reciprocity, typically studied using the Iterated Prisoner's Dilemma (*IPD*), is central to explaining cooperation. In the 1980s Robert Axelrod ran two computer tournaments in which Tit for Tat (*TFT*) emerged as the winner. Yet the archival record is incomplete: for the first tournament only a report survives, and for the second the submitted Fortran strategies remain but not the final tournament code. This gap raises questions about the reproducibility of these historically influential results. We recreate the second tournament by restoring the surviving Fortran implementations to compile with modern compilers and by building an interface that calls the original strategy functions without modification. We reproduce Axelrod's main findings: *TFT* prevails, and successful play tends to be cooperative, responsive to defection, and willing to forgive. Strategy rankings remain mostly unchanged. We then enlarge the field with additional strategies and one of the largest *IPD* tournaments to date, showing the original setting favored *TFT* and that several lesser-known submissions perform strongly in more diverse settings and under noise.

keywords: iterated prisoner's dilemma, evolution of cooperation, replication study, reproducible social science.

Introduction

Cooperation is fundamental to the organization of societies, yet its persistence remains a central puzzle in evolutionary biology and the social sciences. In the absence of supporting mechanisms, natural selection favors defection over cooperation. To explain the emergence of cooperative behavior, several mechanisms have been proposed, among which *direct reciprocity* has played a central role [1–5]. Direct reciprocity relies on repeated interactions among the same individuals, where decisions can be conditioned on a partner's previous behavior. By rewarding cooperation and punishing defection, such interactions provide a pathway for cooperation to be maintained.

The standard model for direct reciprocity is the *Iterated Prisoner's Dilemma (IPD)*. In each round, two players independently decide whether to cooperate (C) or defect (D). Mutual cooperation yields the reward payoff (R), mutual defection gives the punishment payoff (P), and unilateral defection results in the temptation payoff (T) for the defector and the sucker's payoff (S) for the cooperator. With $T > R > P > S$ and $2R > T + S$, the dilemma arises: defection dominates in the one-shot game, even though mutual cooperation would maximize collective welfare.

Because repeated interactions allow for a wide variety of conditional behaviors, direct reciprocity gives rise to a diverse set of strategies. Much of the field has therefore focused on understanding which strategies are "successful" and under what conditions. Researchers have examined which strategies can constitute Nash equilibria [6–9], which can emerge and persist under evolutionary dynamics [3, 10–14], and which perform best in computational tournaments [15–20]. Studies have also sought to identify the principles underlying successful strategies, such as the balance between retaliation and forgiveness, the role of stochasticity [11], or the capacity to adapt to diverse opponents [21].

A pioneering contribution to the study of direct reciprocity and strategies came from Robert Axelrod's computer tournaments in the 1980s [22–24]. Axelrod invited both academics and the wider public, including readers of computer hobbyist magazines, to submit computer programs for playing the *IPD*. Each submission entered a *round-robin tournament*, facing every other entry, a copy of itself, and a random baseline strategy that cooperated with probability 1/2. Average payoffs across all matches determined the final ranking of strategies. The first tournament [22] attracted 14 entries and was won by *Tit for Tat (TFT)*, a remarkably simple strategy that mirrored the opponent's previous move. A second, larger tournament [23] followed, with 63 strategies this time. The main change of the second tournament was that matches no longer had a fixed length but instead had 5 different lengths unknown to the players, removing the possibility of exploiting a known horizon. Despite the new list of opponents, *TFT* again emerged as the winner. Axelrod argued that its success rested on five key properties: it was *nice* (never the first to defect), *provocable* (ready to retaliate), *forgiving* (able to return to cooperation), *non-envious* (not striving to outperform the opponent), and *simple* (easy to understand and implement).

These pioneering tournaments shaped decades of research on cooperation, yet the archival record surrounding them is incomplete. For the first tournament, only the published report survives; the original source code has been lost. For the second tournament, the Fortran implementation of the submitted strategies remains available online (see `http://www-personal.umich.edu/~axe/research/Software/CC/CC2/TourExec1.1.f.html`, accessed on 2025-09-30), but the final code used in Axelrod's reported tournaments has not been preserved.

This highlights a broader challenge of software reproducibility in the *IPD* literature. Over the past 50 years, hundreds of strategies have been proposed, often defined only by fragments of source code, incomplete descriptions, or insufficient supporting data. This has made it difficult to replicate earlier findings, and has led many researchers to evaluate new strategies against small, unrepresentative sets of opponents. Such practices bias conclusions and weaken claims about the relative performance of new strategies.

These challenges are not limited to the *IPD* literature. Reproducibility failures have been widely documented across the social sciences and economics, with large-scale replication projects revealing that only around half of published findings hold up under independent scrutiny [25–27]. Computational research adds further complexity, as analytic flexibility and non-transparent workflows can yield highly variable conclusions even from identical data [28, 29]. Within game theory and related modelling work, the challenge of reproducibility intersects with simulation code, algorithmic implementation, and data provenance. An important step toward addressing this issue has been the `Axelrod-Python` project [30], an open-source Python package that provides a comprehensive framework for implementing and testing *IPD* strategies. The library includes a wide variety of strategies from the literature, together with detailed documentation and usage examples. This project illustrates best practice by providing fully open, tested, and version-controlled artifacts, embodying community principles outlined in reproducibility guides [31–35]. By providing open, executable implementations, `Axelrod-Python` makes it possible to test strategies under common conditions and compare their performance systematically, and it has therefore been used in ongoing research [19, 21, 36–38].

One fundamental question has remained: *can Axelrod's original results be reproduced?* Addressing this question is important not only for assessing the robustness of foundational research on cooperation, but also as a case study in the reproducibility of computational social science more broadly. In this work, we build on the `Axelrod-Python` project to replicate, as faithfully as possible, the outcomes of Axelrod's *second tournament*. Rather than manually re-implementing the strategies from partial descriptions, which would risk introducing errors, we update the surviving Fortran implementations so that they compile with modern compilers, construct an interface that allows them to interact with Python, and integrate them directly with the `Axelrod-Python` library. This design allows us to call the original functions without modification.

Although Axelrod's reported results are impossible to replicate, we are able to recover the main conclusions of Axelrod's study. At the same time, we extend the analysis by incorporating additional strategies from the `Axelrod-Python` library and ultimately conducting one of the largest *IPD* tournaments to date. These extensions shed new light on the robustness of the original findings. Overall, this work makes three contributions. First, it provides the first systematic attempt to reproduce the results of Axelrod's second tournament, which have been central to the study of cooperation. Second, it offers a contemporary perspective on Axelrod's experiments. Finally, by reviving Axelrod's original code and making it accessible through the `Axelrod-Python` library, we preserve for future use the strategies of his second tournament.

Results

Reviving the tournament.

Axelrod's original tournaments cannot be fully replicated. For the first tournament, the only surviving record is the published report; the source code has been lost. For the second tournament, however, the implementations of the 63 submitted strategies remain available. As an example, the Fortran implementation of *TFT* (the implemented function is named `k92r`) is shown in Fig. 1**a**. In this work, we revive the second tournament using these original implementations rather than re-implementing them. To this end, we updated the surviving Fortran code to compile with modern compilers and connected it to the `Axelrod-Python` framework via a Python interface that calls the original functions. The workflow is: (i) build a shared library of the historical strategies; (ii) use the adapter to format the match state, invoke the Fortran routine, and return the action; (iii) let `Axelrod-Python` run matches and assemble tournaments (Fig. 1**b**,**c**). For details on recovering, packaging, and releasing the original source code, see **Methods**.

The major advantage of this approach is that no subjective reinterpretation was introduced. The only modifications were minimal adjustments for compiler compatibility. For several strategies, the Fortran source code is the only surviving description; these strategies are therefore run and evaluated exactly as originally written. This constitutes the most faithful reproduction of Axelrod's work to date.

Reproducing Axelrod's second tournament.

Axelrod's second tournament included 63 strategies. While Axelrod named a few explicitly, most were referred to only by their ranking in the original tournament or by the names of the individuals who submitted them. We now have access to the function names of the Fortran implementations. In what follows, we refer to strategies by their function names, supplemented by Axelrod's names when available. A complete list of the strategies, their authors, their original rankings, the names provided by Axelrod (where available), and their function names is given in the **Supplementary Information** (S1). Notably, the original tournament did not include several strategies that are now considered standard benchmarks in the repeated games literature, such as Cooperator (Always Cooperate), Defector (Always Defect), or Win–Stay–Lose–Shift [39].

Axelrod use the same payoff matrix as in the first tournament, with $T = 5$, $R = 3$, $P = 1$, and $S = 0$. Unlike the first tournament, players were not informed of the number of rounds in advance. In [23] it is stated that:

> *"As announced in the rules, the length of the games was determined probabilistically with a .00346 chance of ending with each given move. This parameter was chosen so that the expected median length of a game would*

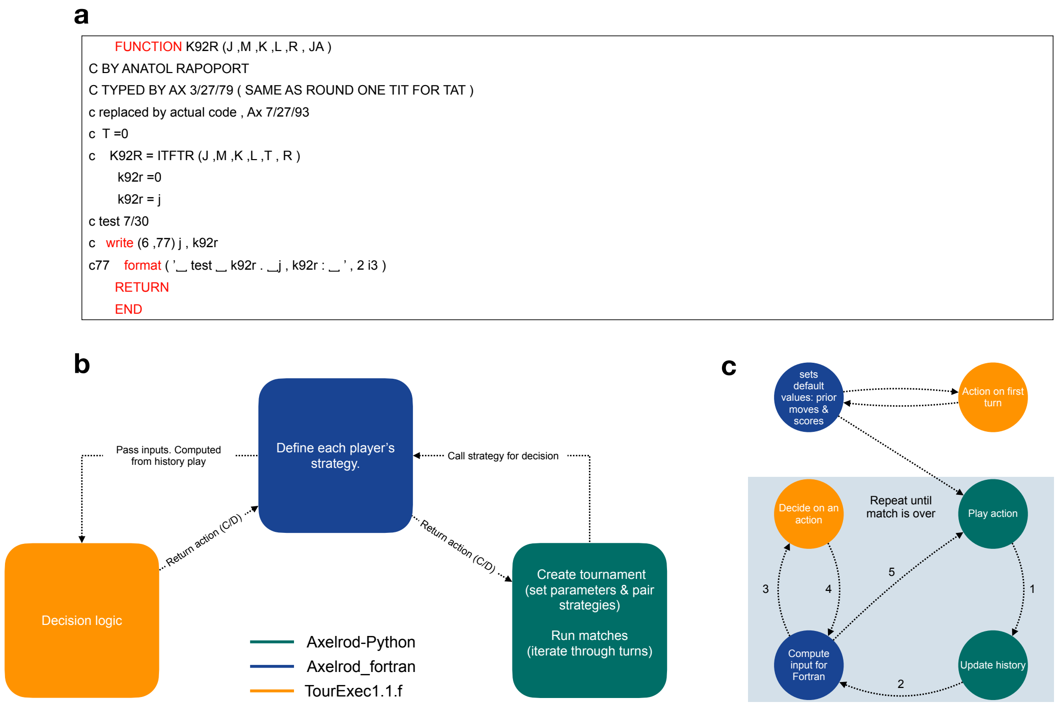

Fig. 1: **Reviving Axelrod's second tournament code. a,** Fortran code for *TFT* (`k92r.f`). Each strategy takes as input the match state (`J`: opponent's previous move; `M`: current turn number; `K`: player's cumulative score; `L`: opponent's cumulative score; `R`: random number in $[0, 1]$; `JA`: player's previous move) and returns 0 (cooperate) or 1 (defect). **b,** Overview of how the three codebases interact. `Axelrod_fortran` creates strategy instances for each player. Tournaments and matches are managed by `Axelrod-Python`, which, at each turn, calls the corresponding `Axelrod_fortran` instance; this, in turn, invokes the original Fortran code to produce a decision from the match history. **c,** Turn-by-turn execution. On the first turn, `Axelrod_fortran` initializes the parameters required by the Fortran implementation, queries it for a decision, and returns the move to `Axelrod-Python`, which plays it and updates the history. On each subsequent turn: (1) the previous outcome is recorded and the history updated; (2) `Axelrod_fortran` accesses the updated history; (3) formats it into the inputs expected by the Fortran code; (4) calls the Fortran function and receives a decision; and (5) `Axelrod-Python` plays the decision. The process repeats until the match ends.

> *be 200 moves. In practice, each pair of players was matched five times, and the lengths of these five games were determined once and for all by drawing a random sample. The resulting random sample from the implied distribution specified that the five games for each pair of players would be of lengths 63, 77, 151, and 308 moves. Thus the average length of a game turned out to be somewhat shorter than expected at 151 moves."*

Thus, termination was not determined probabilistically during play. Instead, Axelrod sampled five match lengths in advance of the tournament and applied the same set to every pair of players. The text lists only four lengths $(63, 77, 151, 308)$ even though five games were played. Using the reported average length of 151 moves, the missing length can be inferred to be 156, so we assume the complete set of match lengths to be $\{63, 77, 151, 156, 308\}$. In **Supplementary Information** (**S10**), we consider the original set of players with a truly probabilistic match length (with an average of 151 turns). We find that this does not lead to any meaningfully different conclusions from the analysis presented here.

It also appears that the only mechanism for averaging out stochastic variation in the original setup was the use of these five fixed repetitions. Without access to the random seed used by Axelrod, it is not possible to reproduce the tournament exactly. To obtain smoother estimates of each strategy's performance, we run Axelrod's second tournament a total of 25,000 times. For each run of the tournament, we use the same five match lengths $(63, 77, 151, 156, 308)$ for every pairing of strategies.

In Fig. 2**a**, we compare the original rankings with those obtained in our replication, with strategies sorted according to their original ranks. As we can see, the winner of the tournament remains *TFT*, while the strategy `k36r`, authored by Roger Hotz, again ranks last. The overall outcomes of most strategies are reproduced, though some do not retain their exact rankings. Notably, the bottom ranking strategies are replicated quite well, consistently performing poorly in both tournaments. Fig. 2**b** shows the difference in rank between the reproduced and original tournaments, ordered from the most negative to the most positive change. We find that 40 out of 63 strategies exhibit a shift in ranking. Most of these changes are modest, typically within 1-3 positions, though two strategies move by 10 places.

These two strategies are `k76r` (originally ranked 46th, now 36th) and `k61r` (originally 2nd, now 12th). The latter, `k61r`, is also known as *Champion*, after its author's surname. Champion cooperates for the first ten moves, plays *TFT* for the next fifteen, and thereafter cooperates unless (i) the opponent defected on the previous move, (ii) the opponent's overall cooperation rate falls below 60%, and (iii) a random draw in $[0, 1]$ exceeds the opponent's cooperation rate. Upon closer inspection, we identified a bug in the Fortran implementation of Champion: an internal variable was not properly initialized. Specifically, the variable `ICOOP` was not assigned an initial value within the function. On the very first call `ICOOP` is effectively treated as 0, but in later matches this assumption no longer holds, meaning that while in the first match `ICOOP` starts at 0 and increases as expected, in subsequent matches its (undefined) value can persist across calls. The corrected version of this strategy is shown in Supplementary Figure 1. It is unclear whether this bug affected the tournament results reported in 1980. One possibility is that each match was run in isolation, which would have prevented the error from arising. In our analysis we use the corrected version of the strategy.

In Fig. 2**c**, we examine the average scores of the strategies. Champion shows a noticeable drop relative to the now second and third ranked strategies. However, its mean score remains very close to that of the rest of the top ten (see Supplementary Table 1). Fig. 2**d** presents violin plots of the score distributions for these top 15 strategies across the 25,000 repetitions. Champion rarely outperforms the second or third ranked strategies, doing so in only a single tournament, confirming that, at least in this version, Champion is not as strong as originally reported.

An interesting observation is that strategies such as `k32r` and `k75r` occasionally achieve the maximum possible score of 2.9. Across all repetitions, these strategies won 0.8% and 2.3% of the tournaments, respectively (Supplementary Table 1). They received little attention in the original tournament, likely because their high variance obscured their potential relative to consistently strong performers such as *TFT*. Another strong and comparatively stable strategy is `k42r`, which won 31.5% of our replicated tournaments; for comparison, *TFT* won 63.3% (Supplementary Table 1). Because the original tournament is a single realization, a different random seed could plausibly have produced a different winner: namely, on another roll of the dice, `k42r` (Otto Borufsen) might well have been crowned the winner.

The source code for `k42r`, together with a description of the strategy's behaviour, is provided in the **Supplementary Information** (S7, Supplementary Figure 5). In summary, the strategy has two states: *normal* and *defecting*. In its normal state, it behaves much like *TFT*, but it includes mechanisms to recover from mutual defection and to escape alternating defections. Every 25 turns, it evaluates the opponent's cooperation rate and switches to defecting mode if the opponent appears random or highly defective. The `k42r` strategy can thus be viewed as a more sophisticated and adaptive variant of *TFT*. These mechanisms are likely intended to prevent endless retaliation and to guard against exploitation by less cooperative opponents. However, in Axelrod's original tournament, `k42r` ranked only 3rd, suggesting that these mechanisms were not particularly advantageous/or used against the original pool of strategies and highly cooperative environment.

We also explored alternative approaches in our attempt to replicate the original ranking (Fig. 2**e**). These include running the tournament with and without self-interactions, as well as testing a Python implementation of Champion that we developed from the textual description in [23]. Across all approaches, Champion consistently exhibits the largest

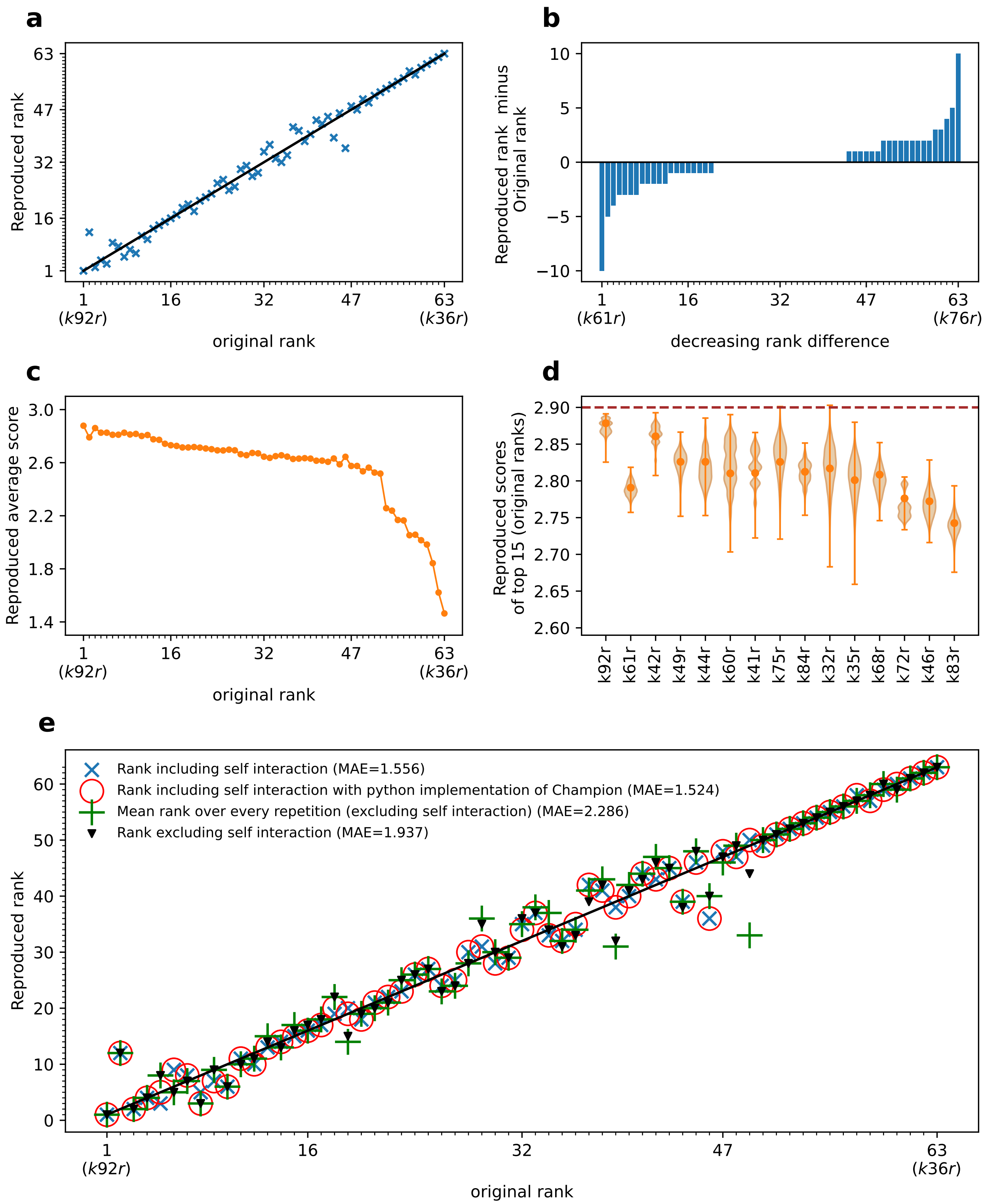

Fig. 2: **Reproducing Axelrod's second tournament.** **a,** Comparison of the original rankings and the replicated rankings. Strategies are ordered by their original ranks; *TFT* again emerges as the overall winner, while `k36r` again ranks last. **b,** Differences in rank between the original and reproduced tournaments, ordered from most negative to most positive change. Most strategies shift by only 1-3 positions, though a few exhibit larger changes, including Champion (`k61r`). **c,** Mean scores of all strategies. Champion's average score drops relative to the other three strategies but remains close to the rest of the top ten. **d,** Violin plots showing the distribution of mean scores for the top 15 strategies across all repetitions (n = 25,000 independent tournament repetitions). For each strategy the orange marker is the mean score and the vertical orange bars span the extrema (minimum to maximum) of the distribution over the repetitions. The brown dashed line marks the maximum attainable mean score of 2.9. **e,** Results under alternative replication approaches, including with and without self-interactions and using a Python re-implementation of Champion. Across all settings, *TFT* remains the winner, while Champion exhibits the largest change in ranking.

change in ranking among the top strategies, while *TFT* remains the overall winner.

Overall, the minor discrepancies in rankings and scores can be attributed to one or more of the following:

- An unknown modification of the source code of Champion (`k61r`) used in the final original tournament code [23].
- Stochastic variation not being sufficiently taken in to account in the original tournament [23].
- A difference with how an older Fortran compiler would interpret the commands, though the implemented version seems to interact as expected.

Axelrod conducted an analysis to identify what he termed "representative strategies" [23]. These were strategies that, according to a linear regression model, served as strong predictors of overall performance (average score). The representative strategies identified in the original tournament were S_6, Reversed State Transition, Ruler, Tester, and Tranquilizer. Using the scores against these five strategies as predictors, Axelrod reported an R^2 value of 0.979, indicating that 97% of the variance in overall performance could be explained by the model. For completeness, the coefficients of this regression (as reported in [23]) are listed in Supplementary Figure 2.

We replicated Axelrod's linear regression analysis and evaluated the predictive power of the original model using our reproduced tournament data. We obtained an R^2 of 0.957, demonstrating that the model continues to be a strong predictor of performance despite the small discrepancies in individual rankings.

To further assess the robustness of the model, we performed a backward elimination procedure to examine how R^2 changes as the number of predictor strategies increases. We find that the largest gain occurs when five predictors are included, after which improvements are small (Supplementary Figure 2.). This suggests that Axelrod's choice of five representative strategies was reasonable. Whether this selection was the outcome of a systematic feature selection procedure or a judgment call remains unclear; in either case, it appears well chosen.

Here we ask: *what was the original tournament like?* The answer is that it was, in fact, highly cooperative. The mean cooperation rate across the tournaments is 75%. When we plot cooperation rates against the original rankings (Fig. 3**a**), most strategies are indeed highly cooperative, with the exception of those in the bottom 14% of the ranking, which display cooperation rates below 0.5.

In Fig. 3**b**, we show that the top performing strategies cooperate almost perfectly with one another, one of Axelrod's key observations. He also emphasized the importance of *provocability* (willingness to punish defection) and *forgiveness* (ability to return to cooperation). In the case of *TFT*, this means responding immediately to the opponent's last move: *TFT* retaliates after defection and resumes cooperation after cooperation. As a result, *TFT* achieves the same cooperation rate as its opponent. To capture this notion, in Fig. 3**c** we plot the difference between pairwise cooperation rates and their transposes, that is, the deviation from symmetry. A value of 0 indicates that two strategies cooperate at exactly the same rate with one another. The top performing strategies cluster near 0, whereas lower performing strategies are more asymmetric, with larger positive or negative deviations.

These outcomes are not surprising. By the time of the second tournament, Axelrod's earlier results were already known, and the strong performance of *TFT* was widely recognized. As a consequence, many newly submitted strategies were explicitly designed to resemble *TFT* or to cooperate effectively with *TFT*-like rules, hence their high levels of cooperation and the symmetry in their interactions.

Revisiting the tournament.

We investigate how the tournament outcomes would change if Axelrod had received additional submissions. To this end, we begin by asking what would have happened had several well-known strategies from the *IPD* literature been submitted

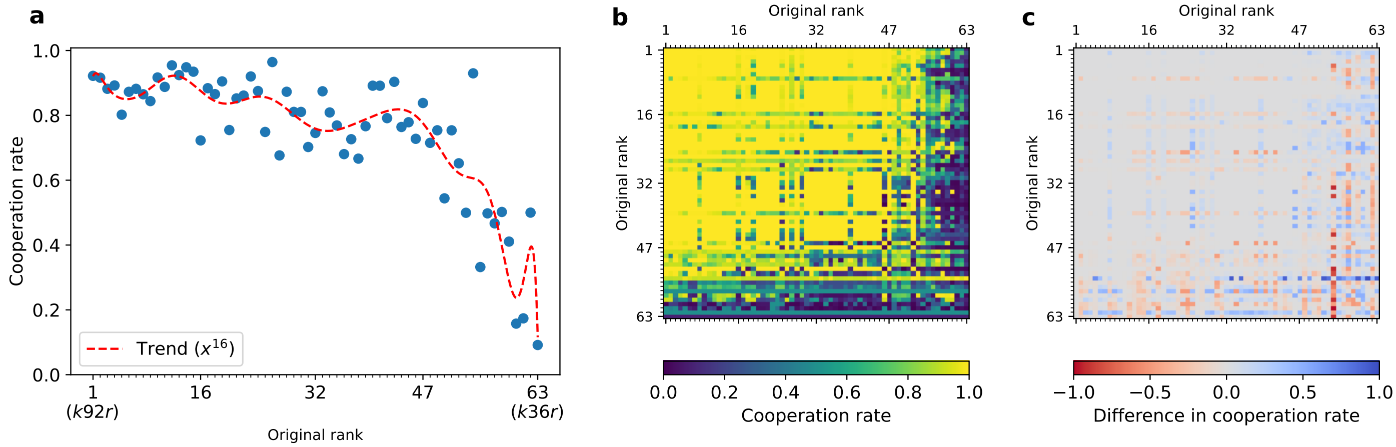

Fig. 3: **Cooperation in Axelrod's second tournament. a,** Mean cooperation rate by original ranking. Most strategies are highly cooperative, with the exception of those at the bottom of the ranking, which fall below 50%. **b,** Cooperation rates between each pair of players, players are ordered by their original ranking. **c,** Deviation from symmetry in pairwise cooperation rates, defined as the difference between cooperation matrices and their transposes. Values close to zero indicate that both strategies cooperate with one another at the same rate.

to the original tournament. We consider classical strategies such as Cooperator, Defector, Win–Stay–Lose–Shift, and Generous *TFT* [40] (*GTFT*), among others. We also include more recent strategies, such as "All-or-nothing-2" [13] (*AON2*) and EvolvedLookerUp2 2 2 [19].

We rerun the tournament using the original set of strategies while adding each of the strategies listed in Table 1. For each tournament, we record the rank of the "newly invited" strategy, while also reporting *TFT*'s rank and the winner of the tournament. Throughout this section, we use Axelrod's original tournament setting, consisting of five repetitions of matches with the lengths described in the previous section. For computational efficiency, each tournament is repeated 250 times.

Several benchmark strategies, including Win–Stay–Lose–Shift and *AON2* (a delayed version of Win–Stay–Lose–Shift), fail to outperform *TFT*. Similarly, EvolvedLookerUp2 2 2, despite its ability to exploit weaker opponents, does not surpass *TFT*. *GTFT* achieves the third highest score overall, but none of these strategies alters *TFT*'s position as the winner of the tournament.

We find that the only strategy capable of changing the outcome of the tournament is Anti Tit For Tat. This strategy begins with cooperation and subsequently plays the opposite of its opponent's previous move. Although simple and seemingly counterintuitive, in repeated games with a finite expected number of rounds it can outperform *TFT*. Importantly, Anti Tit For Tat is another strategy that exploits *TFT*, in the sense that it induces *TFT* to cooperate more frequently than it cooperates itself. *TFT* obtains a payoff of 2.2437 against Anti Tit For Tat, which is not its lowest pairwise payoff. However, the key effect is that interactions with Anti Tit For Tat reduce *TFT*'s overall average payoff while simultaneously leaving Anti Tit For Tat highly vulnerable to exploitation by other strategies. In particular, the more complex strategy `k42r` exploits Anti Tit For Tat especially effectively, achieving an average score of 3.5337 and rising to the top of the tournament. Thus, introducing Anti Tit For Tat has a dual effect: it lowers the average payoff of *TFT*, while simultaneously creating opportunities for exploitative strategies such as `k42r` to achieve exceptionally high scores.

This effect does not persist when all nine additional strategies are introduced simultaneously. Under these conditions, *TFT* benefits from interactions with a broader and more cooperative set of opponents, thereby weakening the disadvantage introduced by Anti Tit For Tat (Table 1).

We next extend this analysis by considering combinations of classical strategies, rather than introducing them individually or all at once. Specifically, we examine all possible subsets of the classical strategy set. For each subset size, we

Strategy	Rank	k92r Rank	Winner	Winner Average Score	Tournament Average Score
AON2	47	1	k92r	2.88019	2.57136
Alternator	58	1	k92r	2.87233	2.55978
Anti Tit For Tat	64	2	k42r	2.87129	2.56407
Cooperator	53	1	k92r	2.88019	2.58131
Defector	64	1	k92r	2.84884	2.52657
EvolvedLookerUp2 2 2	15	1	k92r	2.88019	2.57628
GTFT: 0.33	3	1	k92r	2.88019	2.58122
Two Tits For Tat	28	1	k92r	2.88019	2.57209
Win-Stay Lose-Shift	46	1	k92r	2.88019	2.57409
All Classical Strategies	-	1	k92r	2.84814	2.54673

Table 1: **Effect of adding classical strategies to Axelrod's second tournament.** We examine how the results change when a single strategy from a set of classical strategies from the literature is added to the original tournament. We also consider the case in which all of these strategies are included simultaneously. For each scenario, we report the added strategy, its rank, the rank of *TFT*, the tournament winner, and both the winner's average score and the overall average score of the tournament.

consider every possible combination of strategies that could be added to the original tournament. For example, for subsets of size four, we consider all possible combinations of four strategies selected from the set of nine classical strategies, and for each combination we rerun the tournament with those four strategies added to the original set of 63 strategies. This allows us to evaluate how the performance of *TFT* changes as increasing numbers of classical strategies are introduced, and to identify patterns in how the composition of the tournament affects the outcomes. We also keep track of the performance of `k42r` to provide a broader comparison of how the additional strategies influence tournament dynamics.

In Table 2, we show the winning probabilities of *TFT* and `k42r` as a function of the subset size. For each subset size, the number of possible combinations differs; we denote this quantity by N. We observe that the winning probability of *TFT* displays an approximately symmetric pattern around the midpoint of the subset sizes: it initially decreases and then increases again. This symmetry arises directly from the combinatorial structure of the subsets. In particular, whenever Anti Tit For Tat is included in the tournament, *TFT* ranks second while `k42r` ranks first. As a result, the winning probability of *TFT* is determined entirely by how many subsets contain Anti Tit For Tat. For example, when two new strategies are introduced, there are 36 possible combinations, of which 7 include Anti Tit For Tat. These are precisely the tournaments in which *TFT* fails to win. The same symmetry appears for subset sizes seven and two, six and three, and so forth, producing the symmetric pattern observed in Table 2.

In Figure 4**a**, we show the distribution of the average payoffs achieved by *TFT* and `k42r` across tournaments for each subset size. As the number of possible combinations increases, the diversity of tournament outcomes also increases, leading to broader payoff distributions. At the same time, the average payoffs tend to decrease as more strategies are included. Although the number of tournaments won by `k42r` increases for intermediate subset sizes, the average payoff of *TFT* remains consistently higher, indicating that *TFT* still performs better overall on average. In Figure 4**b**, we additionally show the average cooperation levels across tournaments as a function of subset size. The cooperation distributions become widest for intermediate subset sizes, indicating greater diversity in the resulting interaction structures. Interestingly, these are also the conditions under which the performance of *TFT* is reduced most strongly, suggesting that *TFT* struggles most in environments with highly variable cooperation levels.

The previous analysis focused on a small set of well-known classical strategies, allowing us to systematically investigate how additional invitees alter tournament outcomes. We now extend this analysis to the full `Axelrod-Python` strategy library, substantially increasing both the diversity of opponents and the number of possible tournament configurations.

The `Axelrod-Python` library includes 209 strategies that did not appear in the original tournament. As before, we begin with the original tournament consisting of the 63 submitted strategies and examine what happens when additional

	1 New (N=9)	2 New (N=36)	3 New (N=84)	4 New (N=126)	5 New (N=126)	6 New (N=84)	7 New (N=36)	8 New (N=9)
k92r	0.88889	0.80556	0.75000	0.72222	0.72222	0.75000	0.80556	0.88889
k42r	0.11111	0.19444	0.25000	0.27778	0.27778	0.25000	0.19444	0.11111

Table 2: **Winning probabilities of *TFT* and `k42r` with different combinations of classical strategies.** We consider all possible combinations of the nine classical strategies listed in Table 1. For each subset size, we evaluate every possible combination of strategies added to the original tournament, and we record the winning probabilities of TFT and `k42r`. The number of combinations for each subset size is denoted by N.

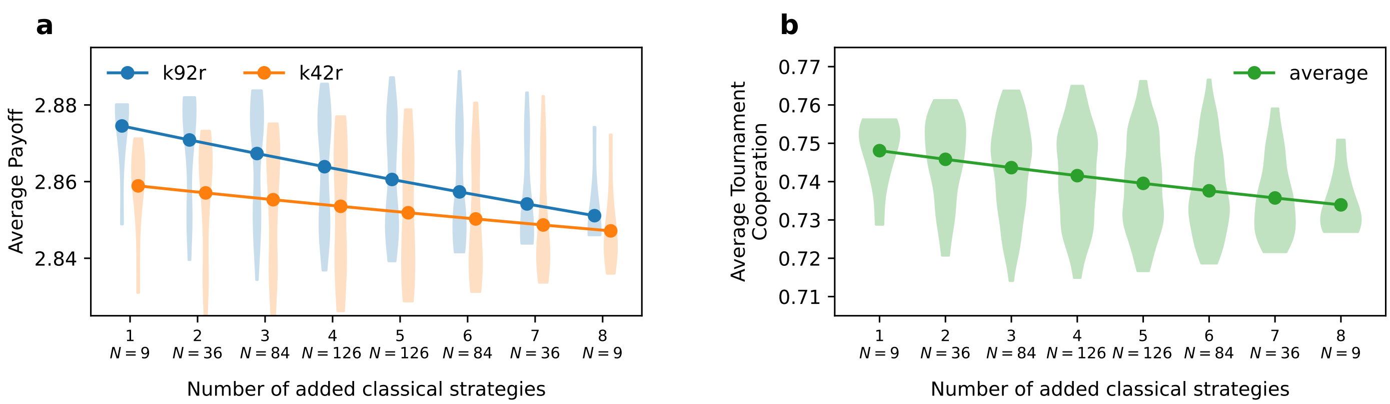

Fig. 4: **Effect of additional classical strategies on average payoffs and tournament cooperation. a,** Distributions of the average tournament payoffs achieved by TFT (`k92r`) and `k42r` as a function of the number of added classical strategies. Violins show the distribution of outcomes across all possible combinations of additional invitees, while lines indicate the mean payoff for each subset size. **b,** Distribution of the average cooperation levels across tournaments for each subset size. Intermediate subset sizes produce the widest cooperation distributions, indicating greater diversity in the resulting tournament interactions. These are also the conditions under which TFT performs least effectively relative to `k42r`. All tournaments use the original tournament setting of five fixed match lengths and are repeated 250 times.

strategies are introduced. For a given subset size, we consider every possible combination of strategies drawn from the 209 additional `Axelrod-Python` strategies and add them to the original tournament.

Due to the combinatorial growth in the number of possible tournaments, we limit our analysis to introducing up to four additional strategies. Even under this restriction, the total number of modified tournaments considered, obtained by summing over all subset sizes from one to four additional strategies, is 78,760,569:

1. Adding 1 strategy: $\binom{209}{1} = 209$ tournaments.

2. Adding 2 strategies: $\binom{209}{2} = 21{,}736$ tournaments.

3. Adding 3 strategies: $\binom{209}{3} = 1{,}499{,}748$ tournaments.

4. Adding 4 strategies: $\binom{209}{4} = 77{,}238{,}876$ tournaments.

Our analysis focuses on how the additional submissions affect the performance of the *original* 63 strategies. For the original strategies that win at least one of these new tournaments, we show the probability of winning the tournament when 1, 2, 3, or 4 additional strategies are included (Table 3). These include *TFT*, and high-scoring strategies such as `k75r`, `k32r`, and `k42r`.

A striking pattern is that the effect of additional invitees on *TFT* remains non-monotonic, even when considering the much larger `Axelrod-Python` strategy library. The probability that *TFT* wins does not simply decrease as more strategies are introduced. Instead, it fluctuates with subset size: adding a single additional strategy increases the probability that *TFT* wins to approximately 85%; with two invitees the probability decreases to approximately 71%; with three invitees it falls further to approximately 60%; and with four invitees it increases again to approximately 75%.

The previous analysis with classical strategies demonstrated that such non-monotonic behaviour can arise from the combinatorial interaction structure generated by particular subsets of opponents. In that setting, a single strategy, Anti Tit For Tat, was sufficient to substantially alter the performance of *TFT* by simultaneously lowering its payoff and creating opportunities for more exploitative strategies such as `k42r`. This suggests that the observed fluctuations are not simply a property of the distribution of strategies, but can instead emerge from the interaction structures induced by specific combinations of opponents.

In the present setting, however, the number of possible tournaments and interaction structures becomes substantially larger, making it difficult to fully characterize the precise mechanisms driving these fluctuations. Nevertheless, the results further highlight the robustness of *TFT* in Axelrod's second tournament: despite the introduction of large numbers of additional strategies, *TFT* remains difficult to unseat as the top performer. The next most successful strategy, `k42r`, remains competitive (e.g., winning up to ∼37% of tournaments with three additional invitees), whereas strategies such as `k32r` and `k75r`, which can achieve high mean scores in some realizations, tend to become less effective as more entrants are introduced.

A natural question is how the results would change if the original tournament had included *noise*, i.e., a small probability that a player's intended action is flipped. Noise was among the main criticisms of Axelrod's study, since *TFT* is not robust to accidental defections [41–44].

Fig. 5 summarizes the results for a noise probability of 0.01. Fig. 5**a** shows rank changes among the 63 original submissions: the left column orders strategies by their original rank, while the right column lists ranks 1-63, with arrows indicating each strategy's rank in the noisy tournament. As expected, noise leads to substantial reordering, with several mid ranking strategies moving into top positions and some former top performers dropping. The mean cooperation rate declines from 0.75 (noiseless) to 0.65 under noise (Fig. 5**c**). Correspondingly, the new winning strategy in this setting cooperates at 0.61, slightly *below* the noisy-tournament mean, whereas *TFT* cooperates at 0.70, *above* the mean. This

	1 New (N = 209)	2 New (N = 21736)	3 New (N = 1499784)	4 New (N = 77238876)
k32r	0.00000	0.00000	0.00001	0.00079
k41r	0.00000	0.00000	0.00001	0.00000
k42r	0.14833	0.26941	0.36640	0.21723
k44r	0.00000	0.00023	0.00057	0.00461
k49r	0.00000	0.00014	0.00035	0.00023
k60r	0.00478	0.01118	0.01882	0.00451
k75r	0.00000	0.00051	0.00245	0.00086
k92r	0.84689	0.71747	0.60814	0.75412
Sum	1.00000	0.99894	0.99674	0.98235

Table 3: **Proportion of wins for original strategies with 1-4 additional invitees.** Shown are the strategies with non zero probabilities of winning at least one of the modified tournaments under five fixed match lengths, repeated 250 times. Additional invitees are strategies drawn from the open-source `Axelrod-Python` library; a full list is provided in **Supplementary Information** (S5 and S8), and source code/documentation are referenced therein. We consider four cases: adding 1, 2, 3, or 4 extra strategies to the original field. The last row reports the sum across these strategies (noting that some wins accrue to strategies not submitted to the original tournament).

pattern reinforces the idea that *cooperating slightly less than the tournament average* is a good predictor of success when noise is present [21]. Figures 5**d** and **e** show that pairwise cooperation becomes more asymmetric and generally lower under noise. Strategies that tend to cooperate less than their co-players perform better. More complex strategies with additional mechanisms also fare well; for example, `k42r`, now ranked 10th, outperforms *TFT*.

We emphasize that evaluating the original submissions under noise is not entirely fair: many authors would likely have incorporated explicit error-correction or forgiveness mechanisms had noise been part of the original rules.

In this section, we examine how the original strategies perform in larger and more diverse tournaments. First, we combine the original submissions with the strategies from the Stewart and Plotkin (S & P) tournament [16]. Next, we run three large-scale tournaments using the full `Axelrod-Python` library under three noise regimes: no noise, 1% noise, and 5% noise. We report the outcomes for each tournament separately. Full results, including all rankings and cooperation rates, are provided in the **Supplementary Information**.

Since the work of Press and Dyson [45], there has been sustained interest in zero-determinant (ZD) strategies. Stewart and Plotkin [16] presented a focused tournament pitting ZD strategies against a small collection of other rules. When all S & P strategies are added to Axelrod's second tournament, the resulting relative rank changes for the original strategies are modest (Fig. 6). The overall level of cooperation remains high (mean ≈ 0.73), and the leading strategies are highly cooperative with one another, mirroring the qualitative pattern of the original tournament. Supplementary Table 3 lists the top-performing strategies (see the **Supplementary Information** (S4) for the full ranking). In this new tournament, ZD strategies are distributed across the ranking rather than concentrated at the top (**Supplementary Information** (S4)), while *TFT*, `k42r`, and generous variants (e.g., GTFT and ZD-GTFT-2) perform particularly well.

The `Axelrod-Python` library contains more than 200 strategies and regularly hosts the largest *IPD* tournaments to date (each time a new strategy is contributed, the results are updated). We expand this tournament further by including all of the original Fortran strategies (omitting the Python duplicates). This produces a tournament with 272 strategies. Supplementary Table 5 shows the top 16 strategies of the tournament.

Rank changes relative to the reproduced tournament are shown in Fig. 7**a**. Several mid performing strategies from the original tournament rise into top positions in this more complex environment. Notably, `k42r` performs best among the original strategies, ranking 16th overall. The cooperation rate of the library-only tournament (excluding the original Fortran strategies) is 0.619. By contrast, the overall cooperation rate of the expanded tournament that includes the

Fortran strategies is 0.626 (see Fig. 7**c**).

Pairwise interactions are shown in Figs. **7d–e**, where top strategies are seen to cooperate strongly with one another, but do not when facing lower ranking opponents. The new winners appear to exploit weaker strategies, which contributes to their success. The highest ranking strategies overall are sophisticated learners trained via reinforcement learning. The "Evolved" strategies were trained with evolutionary algorithms, while the "PSO" strategies were trained using particle swarm optimisation, as described in [36]. These high performing strategies do not necessarily follow Axelrod's original guidelines for success. Instead, they align more closely with the principles identified in [21], which are as follows:

1. Be a little envious.
2. Be "nice" in non-noisy environments or when games are long (if known).
3. Reciprocate both cooperation and defection appropriately: be provocable in short matches and generous in noisy settings.
4. It is acceptable to be clever.
5. Adapt to the environment; adjust to the mean tournament cooperation level.

Finally, we consider the `Axelrod-Python` tournament with noise, using noise probabilities of 1% and 5%. The results are shown in Supplementary Figures 3 and 4. As expected, the overall cooperation rate is lower than in the noiseless, reproduced, and reproduced with noise tournaments. Notably, several of the original submissions perform very well: the best performer from the original set ranks 5th with 1% noise and 3rd with 5% noise. This suggests that some of the more sophisticated strategies submitted to Axelrod's original study were perhaps too complex for the highly cooperative, relatively stable conditions where "cooperate, retaliate after defection, forgive after cooperation, and avoid exploiting others" was optimal. In the presence of errors, and against more complex competitors, these strategies make use of their additional mechanisms to maintain competitive payoffs and robust rankings.

Discussion

Our study revisits one of the most influential computational experiments in the social sciences: Axelrod's second Iterated Prisoner's Dilemma (*IPD*) tournament. By using the original Fortran implementations available online and ensuring that they still compile, we provide the first systematic reproduction of Axelrod's results. Despite minor discrepancies, most notably a bug we identified in *Champion* (`k61r`), our analyses confirm the robustness of Tit For Tat (*TFT*) as a leading strategy. This reinforces Axelrod's conclusion that reciprocity, moderated by the capacity for retaliation and forgiveness, is an effective principle for sustaining cooperation.

By expanding the tournament to include additional strategies from the literature, we reveal clearer patterns. Namely, *TFT* is remarkably robust and difficult to displace when the majority of opponents are drawn from the original pool. In other words, Axelrod's tournament, and similar ones, such as those organized by Stewart and Plotkin [16], create highly cooperative environments in which top performing strategies tend to reciprocate with one another, conditions that are especially favorable to *TFT*.

The picture changes in more diverse or noisy contexts. Several strategies that ranked only moderately in Axelrod's original results rise substantially under increased complexity or noise. A prominent example is `k42r`, which wins 35% of replications of the original tournament in our experiments. This strategy outperforms *TFT* in the presence of noise and surpasses any of the original submissions in tournaments with more elaborate rule sets, such as the full `Axelrod-Python` set of 209 strategies. Other mid performing strategies from the original tournament likewise move into top positions in these settings, particularly when noise is introduced. These findings suggest that some sophisticated designs may have been "over-engineered" for the cooperative, relatively stable conditions of the original experiment, yet prove better adapted to noisy or diverse environments.

This observation aligns with a broader literature showing that *TFT* is not universally optimal. Variants such as Win–Stay–Lose–Shift, *GTFT*, and "All-or-nothing" strategies, as well as "friendly rivals" [7] and strategies discovered via reinforcement learning [19], can outperform *TFT*. Our contribution is to show that, even within Axelrod's original submission set, strategies that received little attention due to modest performance in the original tournament can excel when the environment is more diverse, or noisier. These results also help revise the common impression, shaped by the original tournament, that *TFT* is universally the best performing strategy.

More broadly, our results highlight that tournament rankings in repeated games are inherently shaped by the composition of the participating strategies. Strategies are not evaluated in isolation, but through the interactions that the tournament environment allows them to generate. In the **Supplementary Information**, we compare the interaction structures generated by the original tournament and the `Axelrod-Python` tournament. Specifically, we examine the distributions of cooperation rates and payoffs across pairwise interactions in both settings. In both cases, the interaction distributions are highly skewed towards cooperation. However, the `Axelrod-Python` tournament additionally contains a substantial tail towards low cooperation levels, corresponding to more exploitative interactions. This difference helps explain why *TFT* performs especially well in the original tournament environment. The original tournament is dominated by highly cooperative interactions, which strongly favour reciprocal strategies such as *TFT*. In contrast, the broader `Axelrod-Python` tournament includes a wider range of exploitative interactions, creating environments in which more exploitative or noise-tolerant strategies can become increasingly competitive.

This also raises a broader question regarding the environments in which these strategies are evaluated. To what extent are such tournament environments "objective"? What would constitute an objective tournament environment for evaluating strategies? Should tournaments aim to sample interactions uniformly across behavioural space, or instead balance different forms of reciprocity, exploitation, and cooperation? We believe these are interesting questions for future work.

A second contribution of this work is methodological. To our knowledge, this is the first effort to package and reproduce, according to contemporary best practices, code originally written in the 1980s. The archived materials [46] (at `https://doi.org/10.5281/zenodo.17250038`) are curated to high standards of reproducible research [31–35] and accompanied by a fully automated test suite. All changes to the original code were made systematically and transparently, with complete records available at (`https://github.com/Axelrod-Python/axelrod-fortran`). More broadly, it serves as a case study in computational reproducibility: Axelrod's highly cited, foundational tournaments were built on fragile digital artifacts, of which only fragments survive. Without systematic preservation, much of this historical work might have been lost. By updating, documenting, and integrating the surviving code into an open-source, test-driven framework, we help ensure continued access for the research community.

In conclusion, Axelrod's tournaments remain landmark contributions to the study of cooperation, but their lessons are more context dependent than is often assumed. While *TFT* is strikingly robust in cooperative environments resembling the original tournament, more complex strategies can outperform it in heterogeneous or noisy settings. We hope that the field will continue to develop preservation pipelines that bring historical computational experiments into alignment with modern standards of reproducibility, and to advance reproducible software and science.

Methods

The surviving code of Axelrod's second tournament was written in Fortran and is hosted online by the University of Michigan Center for the Study of Complex Systems [47], where it was last updated in 1996. The source code for all strategies is contained in a single file, `TourExec1.1.f`, embedded in an HTML page.

Our first step was to ensure that this code could be executed with modern software. We stripped the HTML formatting to recover the raw Fortran source and applied only minimal modifications to enable compilation with contemporary Fortran compilers. Each strategy was then extracted into its own modular file. To streamline the build process, we created a dedicated `Makefile` that compiles all strategy files into a single shared library (`libstrategies.so`). Once installed in a standard location on a POSIX-compliant operating system (e.g., `/usr/local/lib/`), this library is automatically located at runtime. The modified source code is openly available at `https://github.com/Axelrod-Python/TourExec` and has been archived at [48], ensuring long-term accessibility.

Each Fortran strategy was implemented as a function with a fixed set of input arguments that encode the state of the match:

- `J`: Opponent's previous move (0 = cooperate, 1 = defect),
- `M`: Current turn number (starting at 1),
- `K`: Player's cumulative score,
- `L`: Opponent's cumulative score,
- `R`: Random number between 0 and 1 (for stochastic strategies),
- `JA`: Player's previous move.

Given these inputs, the function outputs either 0 (cooperate) or 1 (defect).

With the strategies executable again, the next challenge was to run the tournament. For this we rely on the open-source `Axelrod-Python` library, which provides a modern implementation of *IPD* tournaments. In this framework, a *tournament* is a collection of matches, and each *match* is a repeated game between two strategies. A tournament class manages the overall structure (pairing strategies, scheduling matches, aggregating results), while strategy classes determine actions turn by turn based on the recorded history of play.

To integrate the original Fortran strategies into this environment, we developed a Python package, `Axelrod_fortran` [49]. The `Axelrod_fortran` adapter class inherits from the base `Axelrod-Python` strategy class and translates between the two languages: it constructs the required inputs, calls the Fortran function, and returns the chosen move to the Python environment (Fig. 1). The adapter also handles the initial moves of both players required by the Fortran strategies, which is fixed to cooperation in the original code.

Data availability

The data generated and analysed in this study are available in the Zenodo repository [46] at `https://doi.org/10.5281/zenodo.17250038`.

Code availability

The code used to generate and run the tournaments, as well as all analysis scripts, has been archived alongside the data in the Zenodo repository [46] at `https://doi.org/10.5281/zenodo.17250038`, which provides a permanent and citable reference to the exact version used in this study. The code is also accessible via the project's online repository: `https://github.com/Axelrod-Python/revisiting-axelrod-second/`, and is released under the MIT License, an Open Source Initiative approved licence. Instructions for installation and reproduction are provided in the repository README.

Acknowledgements

We thank all contributors to the `Axelrod-Python` library. We also acknowledge the open source ecosystem that enabled this work, including NumPy, pandas, and Matplotlib. This study would not have been possible without the broader open source community.

Funding

This work was not supported by any funding grants.

Author contributions

V.K., O.C., M.H., and N.E.G. designed the study. V.K., O.C., T.J.G., and M.H. wrote the `Axelrod_fortran` software. V.K. and N.E.G. performed the analysis. V.K., M.H., and N.E.G. discussed the results and wrote the manuscript.

Competing Interests

There is no conflict of interest.

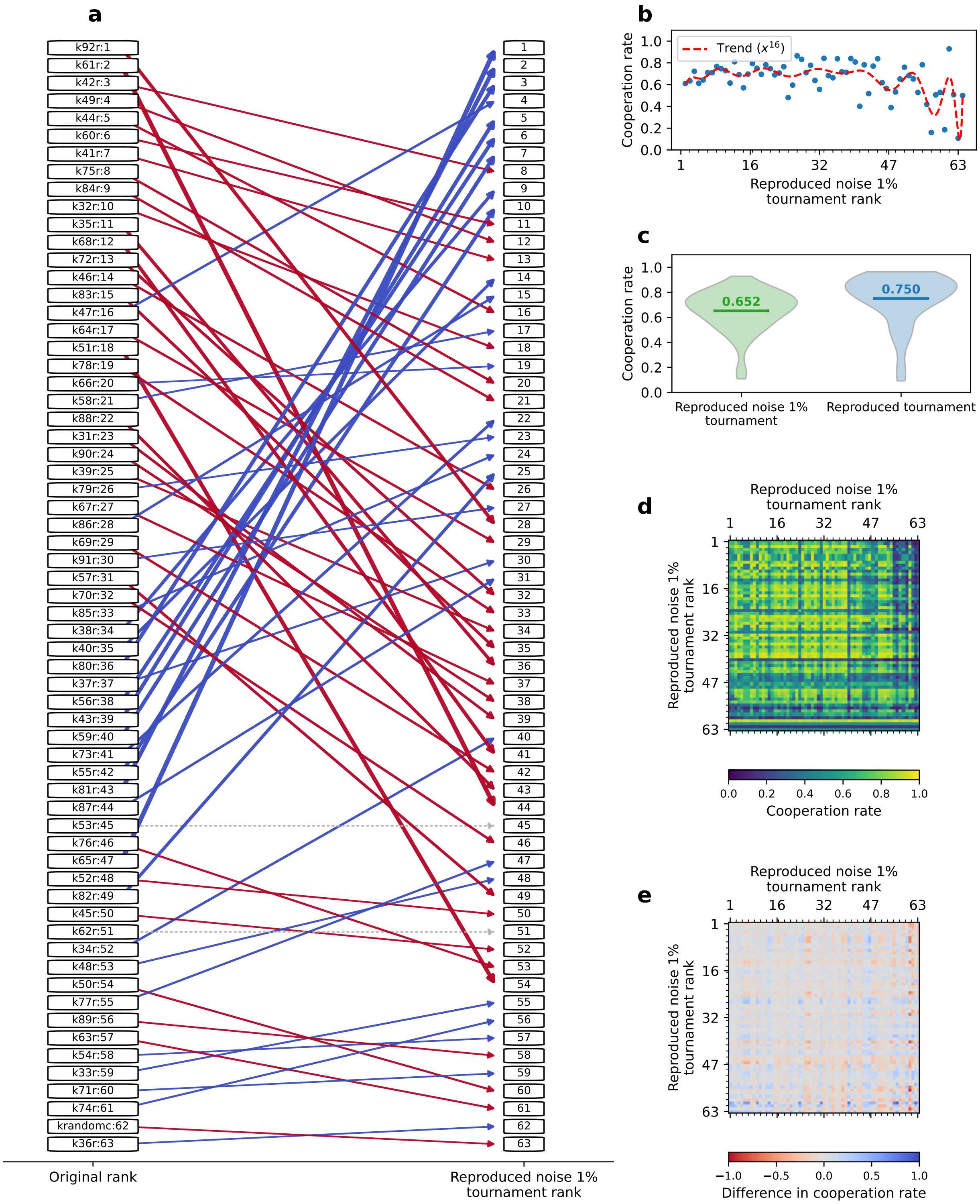

Fig. 5: **Axelrod's second tournament with 1% noise.** **a,** Comparison of the original rankings and the rankings of the second tournament with noise. In the left column, strategies are ordered by their original ranks, and arrows indicate their ranks in the noisy tournament (right column). The arrow widths are proportional to the change in rank. Blue arrows indicate a positive change in rank, red arrows indicate a negative change in rank, and grey arrows indicate no change in rank. **b,** Cooperation rates of strategies ordered by rank in this tournament. **c,** Tournament cooperation rates across all repetitions for this tournament (green) and the reproduced tournament without noise (blue). We also plot the average cooperation rate (green or blue line) and report its value. **d,** Pairwise cooperation rates between strategies. For each pair we calculate the cooperation rate of the row strategy towards the column strategy and vice versa. The strategies are ordered by the ranks in this tournament. For a full list of strategies see Supplementary Table 2, Supplementary Table 4, and **Supplementary Information** (S8). **e,** Differences in cooperation rates for each pair of strategies, again ordered by their rank in the reproduced tournament with noise.

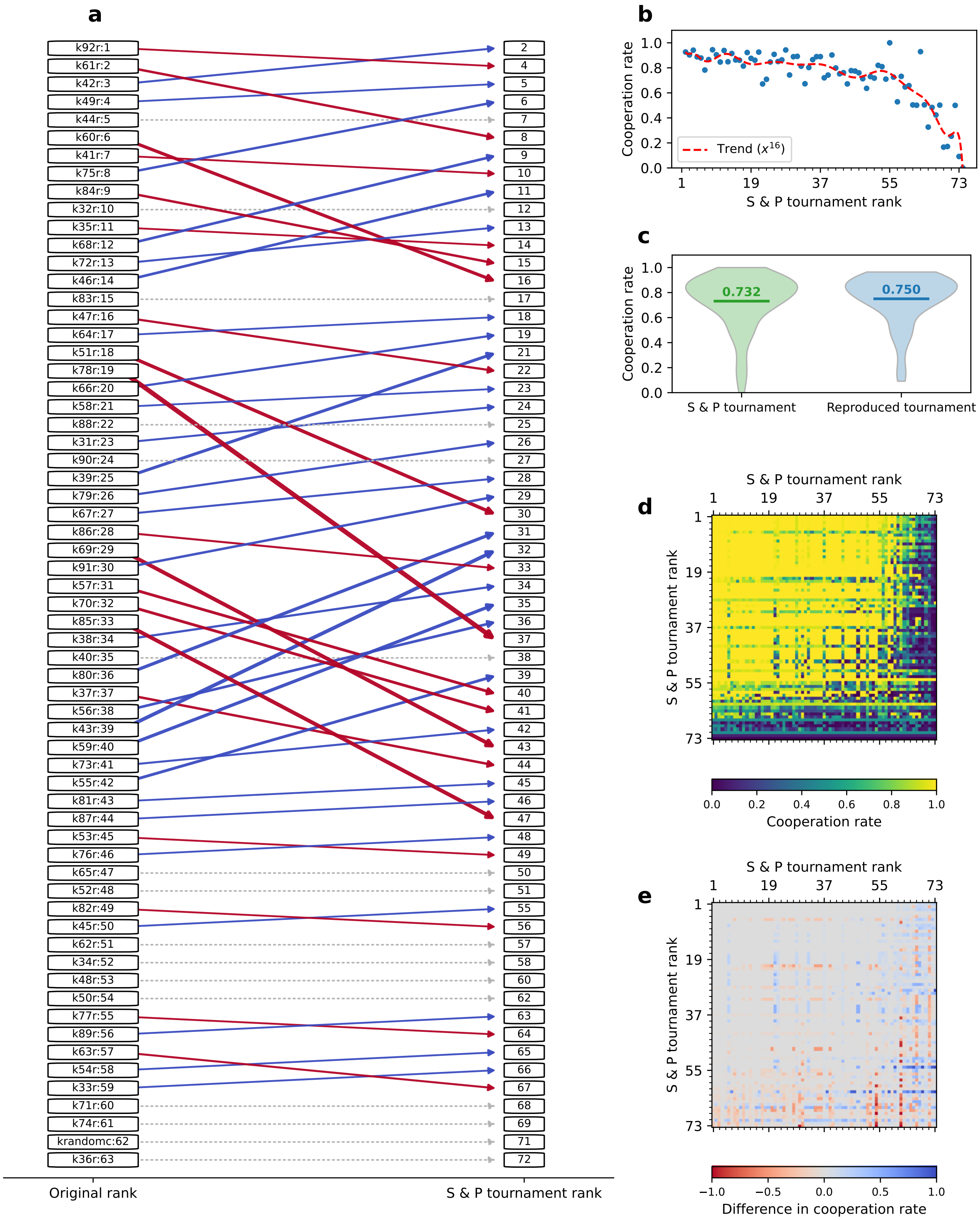

Fig. 6: **Reproduction of Axelrod's second tournament with the addition of the strategies from Stewart's and Plotkin's tournament [16].** The panels follow the same layout as Figure 5. **a,** Comparison of the original rankings (left column) and the rankings in this expanded tournament (right column), with strategies ordered by their original ranks. Arrows indicate the change in rank; arrow widths are proportional to the size of the change. Blue arrows indicate a positive change in rank, red arrows a negative change, and grey arrows no change. **b,** Cooperation rates of strategies ordered by rank in this tournament. **c,** Tournament cooperation rates across all repetitions for this tournament (green) and the reproduced tournament without the additional strategies (blue); the solid line shows the average cooperation rate, whose value is reported. **d,** Pairwise cooperation rates between strategies. For each pair we compute the cooperation rate of the row strategy towards the column strategy and vice versa, with strategies ordered by their rank in this tournament. **e,** Differences in pairwise cooperation rates, again ordered by rank in this tournament.

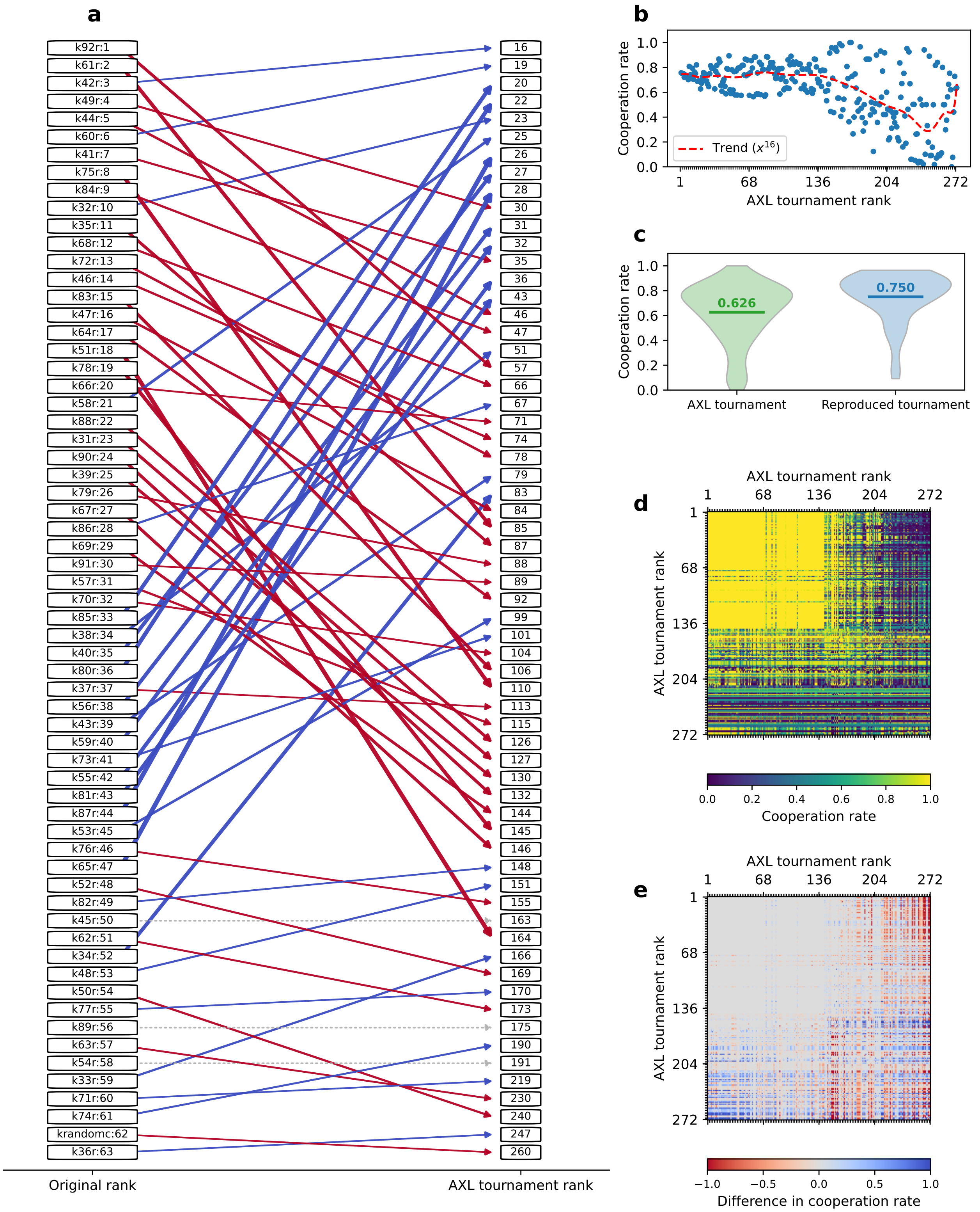

Fig. 7: **Reproduction of Axelrod's second tournament with the addition of the strategies from the `Axelrod-Python` library.** The panels follow the same layout as Figure 6. **a,** Comparison of the original rankings (left column) and the rankings in this expanded tournament (right column), with strategies ordered by their original ranks. Arrows indicate the change in rank, with widths proportional to the size of the change; blue arrows indicate a positive change in rank, red arrows a negative change, and grey arrows no change. **b,** Cooperation rates of strategies ordered by rank in this tournament. **c,** Tournament cooperation rates across all repetitions for this tournament (green) and the reproduced tournament without the additional strategies (blue); the solid line shows the average cooperation rate, whose value is reported. **d,** Pairwise cooperation rates between strategies, ordered by their rank in this tournament. **e,** Differences in pairwise cooperation rates, again ordered by rank in this tournament.

References

[1] Trivers, R. L. The evolution of reciprocal altruism. *The Quarterly Review of Biology* **46**, 35–57 (1971).

[2] Nowak, M. A. Five rules for the evolution of cooperation. *Science* **314**, 1560–1563 (2006).

[3] García, J. & Van Veelen, M. No strategy can win in the repeated prisoner's dilemma: linking game theory and computer simulations. *Frontiers in Robotics and AI* **5**, 102 (2018).

[4] Glynatsi, N. & Knight, V. A bibliometric study of research topics, collaboration and centrality in the field of the Iterated Prisoner's Dilemma. *Humanities and Social Sciences Communications* **8**, 45 (2021).

[5] Rossetti, C. S. & Hilbe, C. Direct reciprocity among humans. *Ethology* **130**, e13407 (2024).

[6] Hilbe, C., Chatterjee, K. & Nowak, M. A. Partners and rivals in direct reciprocity. *Nature human behaviour* **2**, 469–477 (2018).

[7] Murase, Y. & Baek, S. K. Five rules for friendly rivalry in direct reciprocity. *Scientific reports* **10**, 16904 (2020).

[8] Chen, X. & Fu, F. Outlearning extortioners: unbending strategies can foster reciprocal fairness and cooperation. *PNAS nexus* **2**, pgad176 (2023).

[9] Glynatsi, N. E., Akin, E., Nowak, M. A. & Hilbe, C. Conditional cooperation with longer memory. *Proceedings of the National Academy of Sciences* **121**, e2420125121 (2024).

[10] Van Veelen, M., García, J., Rand, D. G. & Nowak, M. A. Direct reciprocity in structured populations. *Proceedings of the National Academy of Sciences* **109**, 9929–9934 (2012).

[11] Baek, S. K., Jeong, H.-C., Hilbe, C. & Nowak, M. A. Comparing reactive and memory-one strategies of direct reciprocity. *Scientific reports* **6**, 25676 (2016).

[12] Amaral, M. A., Wardil, L., Perc, M. & da Silva, J. K. Stochastic win-stay-lose-shift strategy with dynamic aspirations in evolutionary social dilemmas. *Physical Review E* **94**, 032317 (2016).

[13] Hilbe, C., Martinez-Vaquero, L. A., Chatterjee, K. & Nowak, M. A. Memory-n strategies of direct reciprocity. *Proceedings of the National Academy of Sciences* **114**, 4715–4720 (2017).

[14] LaPorte, P., Hilbe, C. & Nowak, M. A. Adaptive dynamics of memory-one strategies in the repeated donation game. *PLoS Computational Biology* **19**, e1010987 (2023).

[15] Li, J. *et al.* How to design a strategy to win an ipd tournament. *The iterated prisoner's dilemma* **20**, 89–104 (2007).

[16] Stewart, a. J. & Plotkin, J. B. Extortion and cooperation in the Prisoner's Dilemma. *Proceedings of the National Academy of Sciences* **109**, 10134–10135 (2012).

[17] Carvalho, A., Rocha, H., Amaral, F. & Guimaraes, F. Iterated prisoner's dilemma-an extended analysis. *Iterated Prisoner's Dilemma-An extended analysis* (2013).

[18] Gaudesi, M., Piccolo, E., Squillero, G. & Tonda, A. Exploiting evolutionary modeling to prevail in iterated prisoner's dilemma tournaments. *IEEE Transactions on Computational Intelligence and AI in Games* **8**, 288–300 (2015).

[19] Knight, V., Harper, M., Glynatsi, N. E. & Campbell, O. Evolution reinforces cooperation with the emergence of self-recognition mechanisms: An empirical study of strategies in the moran process for the iterated prisoner's dilemma. *PloS one* **13**, e0204981 (2018).

[20] Glynatsi, N. E., Akin, E., Nowak, M. A. & Hilbe, C. Conditional cooperation with longer memory. *Proceedings of the National Academy of Sciences* **121**, e2420125121 (2024).

[21] Glynatsi, N. E., Knight, V. & Harper, M. Properties of winning iterated prisoner's dilemma strategies. *PLOS Computational Biology* **20**, e1012644 (2024).

[22] Axelrod, R. Effective Choice in the Prisoner's Dilemma. *Journal of Conflict Resolution* **24**, 3–25 (1980).

[23] Axelrod, R. More Effective Choice in the Prisoner's Dilemma. *Journal of Conflict Resolution* **24**, 379–403 (1980).

[24] Axelrod, R. M. The evolution of cooperation: revised edition (2006).

[25] Open Science Collaboration. Estimating the reproducibility of psychological science. *Science* **349**, aac4716 (2015).

[26] Camerer, C. F. *et al.* Evaluating replicability of laboratory experiments in economics. *Science* **351**, 1433–1436 (2016).

[27] Camerer, C. F. *et al.* Evaluating the replicability of social science experiments in nature and science between 2010 and 2015. *Nature Human Behaviour* **2**, 637–644 (2018).

[28] Silberzahn, R. *et al.* Many analysts, one dataset: Making transparent how variations in analytical choices affect results. *Advances in Methods and Practices in Psychological Science* **1**, 337–356 (2018).

[29] Breznau, N. *et al.* Observing many researchers using the same data and hypothesis reveals a hidden universe of uncertainty. *PNAS* **119**, e2203150119 (2022).

[30] The Axelrod project developers. Axelrod-python/axelrod: v4.10.0. https://doi.org/10.5281/zenodo.3980654.

[31] Wilson, G. *et al.* Best practices for scientific computing. *PLOS Biology* **12**, e1001745 (2014).

[32] Sandve, G. K. *et al.* Ten simple rules for reproducible computational research. *PLoS Computational Biology* **9**, e1003285 (2013).

[33] Wilson, G. *et al.* Good enough practices in scientific computing. *PLoS Computational Biology* **13**, e1005510 (2017).

[34] Stodden, V., Seiler, J. & Ma, Z. Enhancing reproducibility for computational methods. *PNAS* **115**, 2561–2570 (2018).

[35] The Turing Way Community. *The Turing Way: A Handbook for Reproducible, Ethical and Collaborative Data Science* (2022). Zenodo. DOI: 10.5281/zenodo.3233853.

[36] Harper, M. *et al.* Reinforcement learning produces dominant strategies for the iterated prisoner's dilemma. *CoRR* **abs/1707.06307** (2017). URL `http://arxiv.org/abs/1707.06307`.

[37] Bucciarelli, E., Chen, S.-H., Ascatigno, A. & Colantonio, A. Studying the distribution of strategies in the two-scenario snowdrift game. In *Digital (Eco) Systems and Societal Challenges*, 407–428 (Springer, 2024).

[38] Macmillan-Scott, O., Ünver, A. & Musolesi, M. Game-theoretic agent-based modelling of micro-level conflict: Evidence from the isis-kurdish war. *Plos one* **19**, e0297483 (2024).

[39] Nowak, M. & Sigmund, K. A strategy of win-stay, lose-shift that outperforms tit-for-tat in the prisoner's dilemma game. *Nature* **364**, 56–58 (1993).

[40] Nowak, M. A. & Sigmund, K. Tit for tat in heterogeneous populations. *Nature* **355**, 250–253 (1992).

[41] Molander, P. The optimal level of generosity in a selfish, uncertain environment. *Journal of Conflict Resolution* **29**, 611–618 (1985).

[42] Bendor, J. Uncertainty and the evolution of cooperation. *Journal of Conflict resolution* **37**, 709–734 (1993).

[43] Wu, J. & Axelrod, R. How to cope with noise in the iterated prisoner's dilemma. *Journal of Conflict resolution* **39**, 183–189 (1995).

[44] Do Yi, S., Baek, S. K. & Choi, J.-K. Combination with anti-tit-for-tat remedies problems of tit-for-tat. *Journal of Theoretical Biology* **412**, 1–7 (2017).

[45] Press, W. H. & Dyson, F. J. Iterated Prisoner's Dilemma contains strategies that dominate any evolutionary opponent. *Proceedings of the National Academy of Sciences of the United States of America* **109**, 10409–13 (2012). URL `http://www.pnas.org/content/109/26/10409.abstract`.

[46] Knight, V., Campbell, O., Harper, M., Gaffney, T. & Glynatsi, N. E. Source code and data for: Reviving, reproducing, and revisiting axelrod's second tournament. (2025). URL `https://doi.org/10.5281/zenodo.17250038`.

[47] Axelrod, R. Complexity of cooperation web site. http://www-personal.umich.edu/ axe/research/Software/C-C/CC2.html (1996).

[48] Campbell, O. & Knight, V. Axelrod-python/tourexec: v0.3.1 (2017-09-19). https://doi.org/10.5281/zenodo.896461 (2017).

[49] Campbell, O., Knight, V. & Harper, M. Axelrod-Python/axelrod-fortran: v0.3.1 (2017-08-04). https://doi.org/10.5281/zenodo.838980 (2017).